



The Agentic Shift: A Four-Episode Journey into Tool-Using LLMs, Secure Context, and AI-Driven Engineering

Karthik Vaidhyanathan 

From the Editor

If you have been following the explosion of GenAI tools lately, you might have noticed a shift. The initial wow phase of chatbots and magic demos is settling into something more familiar: hard-nosed engineering. We are moving from proof-of-concept prototypes to production, where scaling, security, and performance become critical. In this column, we provide a round-up of our recent episodes that tackle this maturity curve. We hear from Amey Desai on standardization via the Model Context Protocol (MCP), Kacper Lukawski on why serious AI needs specialized vector infrastructure, Sourabh Satish on the reality of securing large language models (LLMs) against prompt injection, and Mark Williamson on how AI is changing and not changing the art of debugging. Together, they provide an overview of an industry that is rapidly building the scaffolding needed to turn stochastic models into reliable software systems.

To hear the full interview, visit <http://www.se-radio.net> or access our archives via RSS at <http://feeds.feedburner.com/se-radio>.—Robert Blumen and Karthik Vaidhyanathan

Episode 689: Amey Desai on the Model Context Protocol (MCP)

Host: Sri Panyam

Guest: Amey Desai, CTO of Nexla

Amey Desai, CTO of Nexla,^a joined host Sri Panyam to discuss the Model Context Protocol (MCP),^b a new standard designed to streamline how large language models (LLMs) connect to external data and tools. Desai began by tracing the friction developers faced post-ChatGPT; while the chat interface was revolutionary, integrating LLMs into actual software resulted in massive amounts of “spaghetti code” and custom scaffolding.

Desai illustrated this friction with a practical engineering example involving the GitHub application programming interface (API). To analyze a team’s performance, a developer typically has to write custom glue code to list users, fetch their pull requests, count reverts, and parse responses for every single step. With MCP, this paradigm shifts from scripting to composition. The developer simply registers these API endpoints as “tools,” and the LLM takes over the orchestration, intelligently sequencing the calls and handling the data flow. Desai noted that this reduces the amount of work one has to reason through just because now you’re looking at a lot fewer lines of code, which is much easier to manage, potentially reducing cognitive load on engineers, allowing them to focus more on business logic. The assumption, however, is that there is an LLM strong to all of this.

^a<https://nexla.com/>.

^b<https://www.anthropic.com/news/model-context-protocol>.



SOFTWARE ENGINEERING RADIO

Visit www.se-radio.net to listen to these and other insightful hour-long podcasts.

RECENT EPISODES

- 688—Daniel Stenberg, Swedish Internet protocol expert and founder and lead developer of the Curl project, speaks with SE Radio host Gavin Henry about removing Rust from Curl.
- 686—François Daoust, W3C staff member and cochair of the Web Developer Experience Community Group, discusses with host Jeremy Jung the origins of the W3C, the browser standardization process, and how it relates to other organizations, like TC39, WHATWG, and IETF.
- 685—Will Wilson, CEO and cofounder of Antithesis, explores deterministic simulation testing (DST) with host Sriram Panyam.
- 681—Qian Li of DBOS, a durable execution platform born from research by the creators of Postgres and Spark, speaks with host Kanchan Shringi about building durable, observable, and scalable software systems.

UPCOMING EPISODES

- Philip Kiely, software developer relations lead at Baseten, speaks with host Jeff Doolittle about multiagent AI, emphasizing how to build AI-native software beyond simple ChatGPT wrappers.
- Mojtaba Sarooghi of Queue-it speaks with host Jeremy Jung about virtual waiting rooms for high-traffic events like concerts and limited-quantity product releases.
- Benjamin Brial, CEO and cofounder of Cycloid, speaks with host Sriram Panyam about internal developer platforms (IDPs) and internal developer portals.
- Taylor Mullen, senior staff engineer at Google and creator of Gemini CLI, joins host Sriram Panyam to discuss the evolution of command-line interface (CLI) based coding agents.

A controversial but compelling point Desai raised was regarding hallucinations. He argued that in the context of orchestration, hallucination should be viewed as a feature, not a bug. It represents the model’s ability to explore a search space, allowing it to try different tool combinations to solve a problem rather than failing on a rigid predefined path, especially in the case of non-critical systems. The conversation

also contrasted MCP with Google’s Agent-to-Agent (A2A) framework.^c Desai described MCP as a “vertical” solution where a single orchestrator manages specific tools, whereas A2A is “horizontal,” allowing agents from different organizations (for example, GitHub talking to Bitbucket) to collaborate. However, he warned of the

^c<https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/>.

mathematical reliability trap in distributed agent systems; if one agent in a chain has 99% reliability, a chain of multiple agents quickly drops challenges in using these protocols.

Desai shared a real-world success story from Nexla involving a PDF orchestrator built to process 10,000 files. Rather than sending every page to an expensive vision model, they used MCP to route simple text PDFs to basic Python libraries and only escalated complex image-heavy pages to the costly LLMs. This “auction bidding” pattern reduced a process that would typically take months to build into a matter of weeks.

However, the outlook wasn’t entirely rosy. Desai warned that security remains the “wild west,” describing prompt injection as “injection on steroids.” He cautioned that giving an LLM access to a file system via MCP is akin to allowing it to execute recursive delete commands if tricked. He advised enterprises to focus on high-value internal use cases like automating release engineering steps before exposing these systems to external risks.

You should listen to this episode if you are tired of writing brittle integration code for your AI applications. Desai provides a clear-eyed look at how protocols like MCP are moving us from a world of custom scripts to intelligent orchestration while realistically assessing the security hurdles that remain.

Episode 691: Kacper Lukawski on the Qdrant Vector Database

Host: Gregory Kapfhammer

Guest: Kacper Lukawski, Senior Developer Advocate at Qdrant

Kacper Lukawski, Senior Developer Advocate at Qdrant,^d joined host

Gregory Kapfhammer to demystify the rapidly growing world of vector databases. Lukawski began by clarifying a fundamental distinction; he prefers the term “vector search engine” over “database.” While traditional databases prioritize ACID properties (atomicity, consistency, isolation, durability), Qdrant prioritizes speed, scalability, and availability to handle high-dimensional vectors generated by neural networks.

The conversation grounded these abstract concepts in the practical shift from keyword search to semantic search. In a traditional keyword search, developers must manually maintain lists of synonyms to ensure that a search for “automobile” matches a document containing “car.” Vector search solves this by converting data, whether text, images, or audio, into numerical embeddings where similar concepts naturally cluster together in mathematical space. Lukawski highlighted that this isn’t just for search bars; it enables applications like anomaly detection (finding vectors that fall far outside normal distributions) and recommendation engines.

A significant portion of the discussion focused on the engineering choices behind Qdrant, specifically the decision to write it in Rust. Lukawski explained that beyond memory safety, Rust was critical for avoiding the garbage collection latency spikes inherent in languages like Java, Go, or C#, which is unacceptable in a high-performance search engine.

Lukawski shared a specific story about performance engineering that illustrates the depth of the problem. Qdrant initially used RocksDB^e for

storage, a standard choice for many database systems. However, they hit a wall with RocksDB’s compaction process, a housekeeping task that reorganizes data, which caused unpredictable latency spikes that blocked the search engine. The team had to engineer a custom solution called *GridStore* to eliminate these spikes while maintaining backward compatibility, a move that significantly smoothed out tail latencies for users.

The episode also addressed a common question for engineers: “Why can’t I just use a vector extension for Postgres (PG Vector)^f?” Lukawski argued that while extensions are fine for small experiments, they hit hard bottlenecks at scale. Vectors are memory hungry; a million OpenAI embeddings can consume 6 GB of RAM, and managing them alongside transactional SQL workloads often degrades the performance of both. Furthermore, specialized engines like Qdrant can perform metadata filtering (for example, “restaurants in New York”) and semantic search in a single pass, whereas extensions often struggle with inefficient prior post-filtering strategies.

You should listen to this episode if you are building retrieval-augmented generation pipelines or search features and are debating between using a general-purpose database extension or a dedicated vector engine. Lukawski provides the architectural arguments needed to make that decision based on scale, latency requirements, and the specific tradeoffs of approximation versus precision.

^d<https://qdrant.tech/>.

^e<https://rocksdb.org/>.

^f<https://github.com/pgvector/pgvector>.

Episode 692: Sourabh Satish on Prompt Injection

Host: Brijesh Ammanath

Guest: Sourabh Satish, CTO and Cofounder of Pangea

Sourabh Satish, CTO of Pangea,^g joined host Brijesh Ammanath to dissect the security vulnerabilities inherent in LLMs, focusing specifically on the top risk identified by OWASP:^h prompt injection. Satish began by distinguishing between consumer and enterprise risks. While a consumer might accidentally expose personal data to a public chatbot, enterprise applications face a magnified risk because they often augment LLMs with sensitive proprietary data, customer records, and internal code.

The core technical vulnerability, Satish explained, stems from the very feature that makes LLMs powerful: their ability to interpret unstructured text and produce human-like text. The model fundamentally cannot distinguish between *system instructions* (the developer's guardrails) and *user input* (potentially malicious commands). They are just a single stream of tokens (tokens are words in simple terms but could be more granular or coarse) to the model.

Satish highlighted the danger of *indirect prompt injection*, using the *Echo Leak* attack as a chilling example. In this scenario, an attacker sends a benign-looking e-mail to a victim. When the victim asks their internal AI Copilot to summarize the e-mail, the AI processes hidden malicious tokens embedded in the message. These tokens might

instruct the AI to extract sensitive data and exfiltrate them by requesting a pixel from an attacker-controlled image server, all without the user realizing it.

To illustrate the difficulty of defense, Satish detailed a Prompt Injection Challenge Pangea hosted with a US\$10,000 prize. The results were stark. In Room 1, where defense relied solely on system prompt instructions (for example, "Do not reveal the secret"), one in five attackers successfully bypassed the guardrails. Attackers used creative techniques like *bookending* (sandwiching malicious commands between distracted instructions) and *multilingual attacks* (using other language characters or Base64ⁱ encoding) to slip past filters designed only for English text.

Satish argued that defense requires more than just better prompts. He advocated for an approach implementing ingress and egress filters that inspect data before they hit the LLM and after they leave. For instance, if an application is for English speakers, an ingress filter should block non-English input entirely. Furthermore, he emphasized that access control must be honored; an LLM should never be allowed to retrieve context data that the authenticated user does not have permission to see in the source system.

You should listen to this episode if you are responsible for securing GenAI applications in an enterprise setting. Satish moves the conversation beyond simple jailbreaks to reveal how sophisticated indirect attacks work and provides a concrete road map for implementing security layers that go beyond trusting the model to behave.

Episode 693: Mark Williamson on AI-Assisted Debugging

Host: Priyanka Raghavan

Guest: Mark Williamson, CTO of Undo

Mark Williamson, CTO of Undo,^j joined host Priyanka Raghavan to explore the intersection of low-level systems engineering and AI-driven workflows. Williamson began by reframing the definition of debugging; it is not merely fixing bugs but rather the quest to understand what your program is doing and why it's not what you expected. He noted that while the industry is obsessed with coding agents, code generation is a minor part of the job; roughly 80% of the productive work is actually self-debugging and understanding complex systems.

The conversation highlighted two primary roles for LLMs in this space: sifting through massive amounts of unstructured data to find patterns and automation to remove repetitive toil. Williamson described the current state of AI debugging as a better rubber duck, one that possesses software engineering expertise and can answer semantic queries like "Why is X null here?" rather than forcing the developer to hunt for keywords.

A major portion of the discussion focused on time travel debugging, a technique that records a program's execution to allow deterministic replay. Williamson distinguished this from traditional debuggers; while a normal debugger acts like a microscope showing *what* is happening at a specific moment, time travel debugging reveals *causality*, allowing a developer to rewind and see exactly *how* a variable became corrupted.

^g<https://pangea.cloud/>.

^h<https://genai.owasp.org/llm-top-10/>.

ⁱ<https://www.base64encode.org/>.

^j<https://undo.io/>.



ABOUT THE AUTHOR



KARTHIK VAIDHYATHAN is with IIIT Hyderabad, Hyderabad 500032, India. Contact the author at karthik.vaidhyathan@iiit.ac.in.

This technology is particularly vital for AI agents. Williamson argued that for an AI to debug effectively without hallucinating, it needs ground truth. Time travel debugging provides this absolute record of reality, allowing an AI to verify its hypotheses against what actually happened rather than guessing.

However, Williamson cautioned that we are in a transitional phase. He humorously compared current autonomous agents, which often try to debug by randomly inserting print statements, to “a Terminator chasing a woolly mammoth and trying to bonk it on the head with a bone.” It

is a juxtaposition of advanced intelligence using primitive tools. He predicted that the MCP will eventually bridge this gap, allowing agents to plug directly into sophisticated debugging tools rather than relying on text manipulation.

You should listen to this episode if you want to look beyond vibe coding and understand how AI can assist with the hardest part of software engineering: understanding legacy code and complex failures. Williamson offers a compelling vision where AI handles the context switching of tooling, allowing the human developer to remain in a pure flow state.

If there is a single takeaway from this collection, it is that the magic of generative AI is rapidly giving way to the discipline of engineering. We are moving past the phase of simple chatbots and vibe coding into an era of robust infrastructure and architectural rigor. Whether it is adopting standardized protocols like MCP to replace brittle integrations, deploying specialized vector engines instead of shoehorning data into relational databases, or erecting rigorous security filters to catch what the models miss, the message is clear; reliance on the model alone is not enough. As Mark Williamson noted, even the smartest agent needs ground truth to be useful. These episodes remind us that while AI can generate the code, it is still the human engineer’s job to build the reliable systems that house it. To hear the full depth of these conversations, the anecdotes, the detailed practices, and the insights that don’t fit on the page, head over to <https://se-radio.net/>. 🎧

IEEE Software (ISSN 0740-7459) is published bimonthly by the IEEE Computer Society. IEEE headquarters: Three Park Ave., 17th Floor, New York, NY 10016-5997. IEEE Computer Society Publications Office: 10662 Los Vaqueros Cir., Los Alamitos, CA 90720; +1 714 821 8380; fax +1 714 821 4010. IEEE Computer Society headquarters: 2001 L St., Ste. 700, Washington, DC 20036. Subscribe to *IEEE Software* by visiting www.computer.org/software.

Postmaster: Send undelivered copies and address changes to *IEEE Software*, Membership Processing Dept., IEEE Service Center, 445 Hoes Lane, Piscataway, NJ 08854-4141. Periodicals Postage Paid at New York, NY, and at additional mailing offices. Canadian GST #125634188. Canada Post Publications Mail Agreement Number 40013885. Return undeliverable Canadian addresses to PO Box 122, Niagara Falls, ON L2E 6S8, Canada. Printed in the USA.

Reuse Rights and Reprint Permissions: Educational or personal use of this material is permitted without fee, provided such use: 1) is not made for profit; 2) includes this notice and a full citation to the original work on the first page of the copy; and 3) does not imply IEEE endorsement of any

third-party products or services. Authors and their companies are permitted to post the accepted version of IEEE-copyrighted material on their own web servers without permission, provided that the IEEE copyright notice and a full citation to the original work appear on the first screen of the posted copy. An accepted manuscript is a version that has been revised by the author to incorporate review suggestions, but not the published version with copyediting, proofreading, and formatting added by IEEE. For more information, please go to: http://www.ieee.org/publications_standards/publications/rights/paperversionpolicy.html. Permission to reprint/republish this material for commercial, advertising, or promotional purposes or for creating new collective works for resale or redistribution must be obtained from IEEE by writing to the IEEE Intellectual Property Rights Office, 445 Hoes Lane, Piscataway, NJ 08854-4141 or pubs-permissions@ieee.org. Copyright © 2026 IEEE. All rights reserved.

Abstracting and Library Use: Abstracting is permitted with credit to the source. Libraries are permitted to photocopy for private use of patrons, provided the per-copy fee is paid through the Copyright Clearance Center, 222 Rosewood Drive, Danvers, MA 01923.